

PROJECT DEVELOPMENT PHASE

Performance Testing

Date	01 October 2026
Team ID	SWTID-2026-4057
Project Name	Implement Client Script & UI Policy (Incident)

1. Performance Testing Objective

The objective of performance testing for this project is to verify that the configured UI Policy and Client Scripts respond appropriately during normal Incident form and list interactions without introducing unnecessary user-interface delay. The testing focuses on the implemented client-side controls rather than server-side load or infrastructure benchmarking.

2. Performance Areas

Area	What is Checked	Expected Behavior
UI Policy response	Behavior when Impact is changed to 1 - High and when the condition becomes false.	Field controls are applied or reversed during normal interaction.
onChange response	Impact change triggers the Urgency update and information message.	Urgency is set to 1 - High without requiring form submission.
onSubmit validation	Save attempt with High Impact and empty Assigned To.	Validation executes before the record is saved and displays the error.
onCellEdit response	Direct State editing from the Incident list.	The edit is rejected and the user receives the configured alert.
Normal interaction	Valid Incident form updates under normal conditions.	Valid changes remain usable without unnecessary blocking.

3. Performance Test Scenarios

Test ID	Scenario	Action	Expected Result
PT-01	High Impact UI Policy	Set Impact to 1 - High.	Assignment group becomes mandatory and Urgency becomes read-only.
PT-02	Automatic Urgency update	Change Impact to 1 - High.	Urgency changes to 1 - High and the information message appears.
PT-03	Save validation	Attempt to save with High Impact and Assigned To empty.	Save is blocked and an error is displayed on Assigned To.
PT-04	Valid save	Provide Assigned To and save the High Impact Incident.	The record can be saved successfully.
PT-05	Reverse behavior	Change Impact from High to another value.	High Impact controls are reversed; normal field behavior returns.
PT-06	List edit restriction	Edit State directly from the Incident list.	The edit is blocked and the configured alert is displayed.
PT-07	Form State update	Open the Incident and update State through the form.	The form-based State update remains available.

4. Performance Acceptance Criteria

- Client-side controls should respond during the relevant user action without requiring an unnecessary form submission.

- Automatic field updates should occur when the corresponding condition is triggered.
- Validation should execute before an invalid save is accepted.
- Restricted list editing should be stopped immediately by the onCellEdit control.
- Normal form interaction should remain available when the configured conditions are not active.
- No performance issue should be introduced by repeated execution of the implemented client-side controls during normal use.

5. Test Result Recording

Test ID	Observed Result	Status
PT-01	High Impact controls applied as configured.	Pass
PT-02	Urgency automatically set to 1 - High.	Pass
PT-03	Invalid High Impact save prevented.	Pass
PT-04	Valid Incident update accepted.	Pass
PT-05	Normal behavior restored after reversing condition.	Pass
PT-06	Direct State list edit blocked.	Pass
PT-07	State can be updated through the Incident form.	Pass

6. Conclusion

The performance-oriented checks confirm that the implemented Incident controls are designed to operate at the point of user interaction: UI Policy behavior changes with the Incident condition, the onChange script updates Urgency, the onSubmit script validates before saving, and the onCellEdit script prevents direct State list editing. This document records functional responsiveness expectations for the implemented client-side configuration; it does not claim a separate infrastructure or large-scale load benchmark.